# 高速處理器為何需要記憶體階層

Processor 每 cycle 可以執行運算，DRAM 存取卻可能需要數百個 processor cycles。若每次 load/store 都直接等待 DRAM，快速 datapath 會長時間閒置。

Memory hierarchy 用不同容量、成本與延遲的儲存層共同提供「近似快速且近似無限」的記憶體幻象。它依賴 locality（局部性），並不保證任意程式都得到相同效能。

本章依序建立 cache、virtual memory、TLB、virtual machine 與 coherence。每個結論都必須附上容量、block/page、miss penalty 與 write policy。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 位址、page 與 cache block 的單位

本章會重複使用四種位址：

1. VA（virtual address，虛擬位址），process 執行 load/store 時使用。
2. VPN（virtual page number，虛擬頁號），VA 去掉 page offset。
3. PA（physical address，實體位址），page translation 後存取 main memory 或 physically addressed cache。
4. Cache block address，PA 去掉 block offset。

Byte address 的最小單位是 1 byte。以本章 4-byte word 為例，word index 乘 4 才轉成 byte offset；byte address 除以 block size 才得到 block number。手算若混用單位，看似相同的低位元會得到錯誤 mapping。

本章的 $B$ 表示 block size bytes，$P$ 表示 page size bytes。公式會在首次使用時重新定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.1 局部性與記憶體階層成本

Storage hierarchy 通常由上到下為 registers、L1 cache、L2/L3 cache、main memory、flash/disk 與遠端儲存。越上層越快、每 byte 越貴且容量越小；越下層越慢、越便宜且容量越大。

理想是讓常用資料靠近 processor，低頻資料留在下層。Locality 是此設計成立的經驗前提，不是硬體能對任意程式保證的定理。

Temporal locality（時間局部性）表示近期用過的位置很可能近期再用。Spatial locality（空間局部性）表示使用某位置後，附近位置很可能很快被使用。

以連續 4-byte 整數陣列為例，依序讀取 byte addresses 0、4、8、12、16。假設 cache 原本為空、block 為 16 bytes、沒有 prefetch 且過程中不發生 eviction：前五次依序是 miss、hit、hit、hit、miss；前三個 hit 利用同一 block 的空間局部性。若隨後再次讀位址 16 且該 block 仍在，這次 hit 則是時間局部性的例子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 為何 cache 能改善平均時間

設上層 hit time 為 $H$，miss rate 為 $m$，miss penalty 為 $P_m$（在已付 hit lookup 時間後額外等待的時間）。平均記憶體存取時間（AMAT）為

$$\mathrm{AMAT} = H + mP_m.$$

若 $H = 1$ cycle、$m = 0.05$、$P_m = 80$ cycles，則

$$\mathrm{AMAT} = 1 + 0.05 \times 80 = 5 \text{ cycles}.$$

即使 miss 很慢，95% 的 accesses 可以由 1-cycle cache 滿足。若 hit rate 降至 80%，AMAT 變為 17 cycles，顯示 low hit rate 如何吞掉 cache 的價值。

Miss rate 的分母必須一致。Instruction miss rate 常以 instruction 為分母，data cache miss rate 常以 data references 為分母。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Memory wall 與資源取捨

Processor performance 與 DRAM latency 的改善速度不一致，形成 memory wall。提高 capacity、associativity 或 block size 可能降低某些 misses，但也可能增加 hit time、miss penalty、面積與 power。

一個 hierarchy 設計是多目標最佳化：

- L1 優先降低 hit time，並避免 drag critical path。
- 下層 cache 承受較大容量與較高 associativity。
- Main memory 提供容量與 bandwidth。
- Disk/remote storage 處理容量與持久性，但 miss cost 遠高於 cache。

「增加 cache 一定更快」缺少 workload 與 hit-time 條件；容量增加可能使 index/select path 變慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.2 Cache 的基本模型

Cache block（或 line）是 cache 與下層之間傳輸的單位，通常包含多個 words。Cache entry 至少保存 data、tag 與 valid bit；write-back cache 另外保存 dirty bit。

一次 reference 的步驟是：

1. 用 address 的 index 選 set。
2. 比較 set 內各 way 的 tag 與 valid。
3. 若 match，回傳 block 內由 offset 指到的 byte/word。
4. 若 miss，向下一層取得 block，選擇 victim，再寫入 cache。

Valid 防止未初始化 bits 因 tag 恰好相同而被誤認為 hit。

Tag 相同但 valid 為 0 仍是 miss；dirty bit 只表示是否需 write-back，不用來判斷 hit。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Direct-mapped 映射

Direct-mapped cache 的每個 block 只能放在唯一位置：

$$\text{cache block index} = \text{memory block number} \bmod N_{\text{blocks}}.$$

若 cache 有 $2^q$ blocks，index 是 block number 的低 $q$ bits。Offset 選擇 block 內位置，tag 保存剩餘高位。

若兩個 frequently used blocks 映射到同一 index，即使 cache 其他位置空著，也會互相 evict。這是 conflict miss 的典型來源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Accessing a cache：一個具體 trace

假設 4-block direct-mapped word cache，block size 與 word 相同。Block addresses `0,1,2,3,0,4,0` 的存取為：

| Access | Block | Index | 結果 | 原因 |
|---|---|---|---|---|
| 1 | 0 | 0 | miss | valid=0 |
| 2 | 1 | 1 | miss | valid=0 |
| 3 | 2 | 2 | miss | valid=0 |
| 4 | 3 | 3 | miss | valid=0 |
| 5 | 0 | 0 | hit | tag/valid match |
| 6 | 4 | 0 | miss | evict block 0 |
| 7 | 0 | 0 | miss | conflict with block 4 |

# Accessing a cache：一個具體 trace （續）

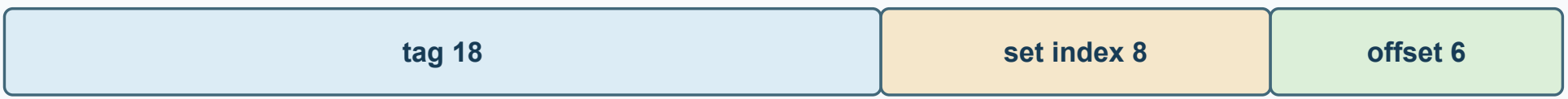
七次有六次 miss，miss rate 為 $6/7$。前四次與第一次碰到block 4的第六次屬compulsory；第七次對block 0的重訪是conflict，因同容量fully associative LRU在第六次會淘汰block 1而保留剛用過的block 0。

每次 miss 都要列 victim 與 valid/tag/dirty 更新，不能只算 hit count。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Address decode：64 KiB、4-way、64-byte block

## 64 KiB、4-way、64-byte cache 的位址切分




**0x12345678**

tag=0x48d1

set=0x59 = 89

offset=56

**1,024 blocks**

256 sets

4 ways / set

64 bytes

先以 byte address 定位 block，再切 set 與 offset；全容量必須扣除非資料欄位後另行核對。

$$N_{\text{blocks}} = 2^{16}/2^{6} = 1024, \quad N_{\text{sets}} = 1024/4 = 256.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Address decode：64 KiB、4-way、64-byte block （續）

因此：

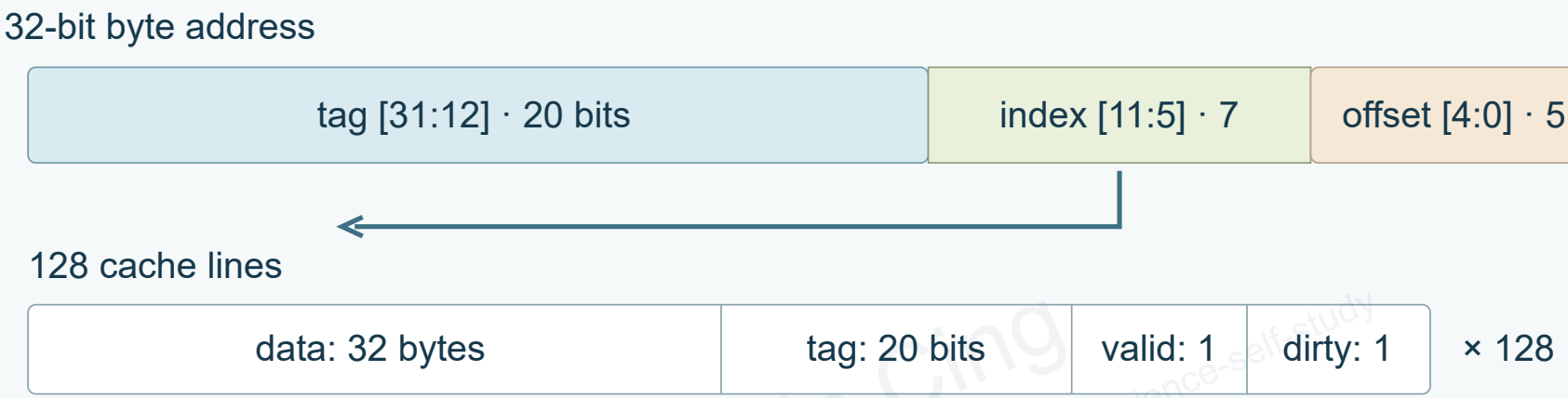
- block offset bits：$\log_2 64 = 6$。
- set index bits：$\log_2 256 = 8$。
- tag bits：$32 - 6 - 8 = 18$。

Address `0x12345678` 的低 6 bits 是 `111000`，偏移 56 bytes；接著 8 bits 是 `01011001`，index 為 89；高位是 tag `0x48d1`。

同樣 block 內的 `0x12345658` 有相同 tag/index、不同 offset，所以前 block 若仍 valid，這次應為 hit。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 從位址欄位反推容量與額外成本




以 32-bit byte address 的 direct-mapped write-back cache 為例：offset 5 bits 表示每 line 有 32 bytes；index 7 bits 表示共有 128 lines，故資料容量為 128 × 32 = 4096 bytes。

每列還存 20-bit tag、1 個 valid 與 1 個 dirty，共 22 bits；全 cache 的額外成本為 128 × 22 = 2816 bits = 352 bytes。只計上述欄位時，總儲存為 4448 bytes，是 data capacity 的 1.0859375 倍；ECC 與其他控制電路另計。

# Handling cache misses

Simplified in-order processor 的 read miss 通常會 stall。Controller 要保存 miss address、送 read request、等待下層、選 victim、更新 data/tag/valid，再讓 instruction 重試。

Instruction miss 的重試 PC 是發出 request 的 instruction address，不是 PC 已被更新後的值。Data miss 要保存 load/store 所需 address 與 store data。

Write-back dirty victim 必須先 write-back 或放入 write-back buffer，再 fetch new block。Critical-word-first 可以先把 requested word 送回 processor，再完成 block fill。

Cache miss 通常是 pipeline stall 與 controller transaction；page fault 會經 exception 進入 OS，恢復成本與 state 處理不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Memory system 如何支撐 cache

Block fill 的 miss penalty 可拆成 address transfer、DRAM latency、burst transfer 與 cache update。以 four-word block、one-word-wide memory 為例，若 address 1 cycle、每次 DRAM access 15 cycles、每 word transfer 1 cycle：

$$1 + 4(15) + 4(1) = 65 \text{ bus cycles.}$$

Wider memory/bus 可平行取兩個 words，總時間可降為

$$1 + 2(15) + 2(1) = 33 \text{ cycles.}$$

這個例子假設 burst 次數依 memory width 減少，且 transfer 花費依 bus width 降低；不同 interconnect 要重新建模。

Block size 增加可提高 spatial locality 利用，卻也可能放大 miss penalty 與污染 cache。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 交錯式記憶體：讓不同 bank 同時工作

四個連續 words 分別放在四個獨立 bank；假設一個共用地址操作在 cycle 1 同時啟動四個 bank，各 bank 延遲 15 cycles，而匯流排每 cycle 傳一個 word。資料都就緒後，再依序傳四個 words：總 fill time = 1 + 15 + 4 = 20 bus cycles。

比較同一模型下單 bank 的 65 cycles：收益來自重疊 bank latency，而不是一次在窄 bus 上傳四個 words。若位址無法同時發出、連續資料映到同一 bank，或 bank 回應有額外間距，20 cycles 就不成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Handling writes

Write-through 同時更新 cache 與下層。它簡化 coherence 與 recovery，但每次 store 都消耗 lower-level bandwidth；write buffer 可讓 processor 不必逐筆等 memory。

Write-back 只更新 cache，並設 dirty bit；victim 被 evict 時才寫回。它降低連續寫入的 traffic，但 eviction、coherence 與 recovery 更複雜。

Write-allocate 在 write miss 時先 fetch entire block；no-write-allocate 直接把 write 傳給下層。Policy 可以依 workload、頁面或 instruction 改變，例如整頁清零時不必先 fetch。

小於 cache line 的 store 只更新指定 byte；實作可使用 byte enable 或其他等效機制，不必然執行 read-modify-write，也不必然讀取 main memory。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Write policy 的邊界案例

| 策略 | Write miss 行為 | 優點 | 主要代價 |
| --- | --- | --- | --- |
| write-through + no-allocate | 直接寫下層 | 邏輯簡單 | 下層 traffic 高 |
| write-through + allocate | fetch block，之後同步寫下層 | 後續 reads 可能 hit | write bandwidth 高 |
| write-back + allocate | fetch block，僅標 dirty | 連續 writes 省 traffic | dirty eviction |
| write-back + no-allocate | 直接寫下層 | 避免無用 allocation | 仍失去後續 write hit |

表中「allocate」是 placement policy，「through/back」是 lower-level update policy，兩者不可混為一組。

Store buffer 若保存未完成 writes，read-after-write 要看該 address 是否被 buffer 遮住，不能盲目由 cache 回舊值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

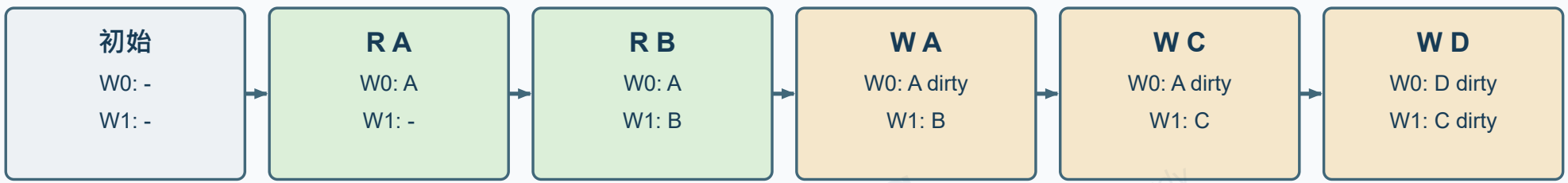
# Write-back、write-allocate 與 dirty victim trace

考慮 2-way set-associative，64-byte blocks，每次 store 只改4 bytes，LRU replacement，write-back、write-allocate。每個 set 初始沒有 valid entry，空 way 選編號較小者。Way 是固定實體槽位，更新 LRU 不搬動其他 line。以下 `A/B/C/D` 均映射同一 set；`dirty` 代表 cache copy 比下層新。

| Step | Access | Way 0 | Way 1 | Victim 與 lower-level action |
|---|---|---|---|---|
| 0 | initial | invalid | invalid | 無 |
| 1 | read A | A clean | invalid | miss；read A 一個 block |
| 2 | read B | A clean | B clean | miss；read B |
| 3 | write A | A dirty（MRU） | B clean | write hit；不下層 |
| 4 | write C | A dirty | C dirty（MRU） | miss；B clean 可丟；read C 後寫 C |
| 5 | write D | D dirty（MRU） | C dirty | miss；A 是 LRU 且 dirty，先 write-back A，再 read D |

# Write-back、write-allocate 與 dirty victim trace （續）

## 2-way write-back cache：LRU replacement 與 dirty victim




**W C evicts clean B；W D evicts dirty A，必須先 write-back 0x...A 才能安裝 D。**

**Lower traffic：read A/B/C/D = 4 blocks；write-back A = 1 block；write-hit A 不立即送下層。**

這個 trace 下層共讀 4 blocks（A、B、C、D），共寫 1 個舊 block A。若控制錯誤地在 `Allocate D` 時先覆寫 A，A 的最新值只存在 cache，之後無法恢復；dirty bit 不影響 hit detection，卻決定淘汰前是否必須保存最新資料。Cache 與 DRAM 仍可能在斷電時失去內容；正確 write-back 不等於持久化保證。

# Write-back、write-allocate 與 dirty victim trace （續）

換成 write-through、no-write-allocate：

1. Step 3 的 write A 立即送往下層。
2. Step 4 的 write C 不配置 cache，直接送下層，B 不會被 eviction。
3. Step 5 的 write D 也直接送下層。
4. Set 仍只保留 A、B；下層 reads 是 A、B，writes 是 A、C、D。

只計 payload bytes，不含 headers/acknowledgments，也不在序列末尾強制 flush。WB/WA 的4個64-byte fill加1個64-byte write-back為320 bytes；WT/NA的2個64-byte fill加3個4-byte stores為140 bytes。兩者都有5次下層動作，卻不是相同大小的5個block transfers。WB最終C、D仍dirty，若要求再寫回下層，需另加128 bytes；持久性仍需平台的flush及ordering承諾。

Correctness argument：write hit 必須在同一個 cache block 的 store 完成前保持 exclusive ownership；write-back 只延後 memory update，不延後 cache 的 program-order value。Eviction 只有在 victim data 已送往 memory、write buffer 或 other owner 且該 transfer 有完成保證後，才能重用 victim storage。

# FastMATH cache 案例

本章案例的 Intrinsity FastMATH 是 MIPS embedded processor 教學案例。它使用 separate instruction and data caches，讓 pipeline 在同一 cycle 取 instruction 與存取 data；每個 cache 為 16 KiB、4K words、16-word blocks。

以 4K words 且每 block 16 words 計算，共有 256 blocks。若 architecture 的 cache 是 direct-mapped，index 有 8 bits、offset 有 6 bits；詳細實作仍以該處理器圖與規格為準，不能只由容量猜出全部 timing。

Separate caches 消除 instruction fetch 與 data access 的 structural conflict，但 instruction/data traffic 不再共享同一 capacity。

歷史實例的 cycle numbers 是模型輸入，換到今日 processor 必須重測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.3 Cache performance

Processor time 可寫成

$$\text{CPU time} = (CPU_{\text{exec}} + CPU_{\text{stall}}) \times T_{\text{clock}}.$$

若每次 L1 miss 固定 penalty $P_m$，instruction miss rate 為 $m_i$，data references per instruction 為 $f_d$，data miss rate 為 $m_d$：

$$\text{CPI} = \text{CPI}_{\text{base}} + m_i P_m + f_d m_d P_m.$$

這是 in-order、每次 miss 全 stall 的簡化模型。Out-of-order processor 可重疊部分 misses，需用 stall distribution 或模擬。

不要先把 instruction 與 data miss rate 平均成單一 miss rate；兩類 request 的分母不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 分離式與合併式 cache：先對齊分母

考慮 100 條退休指令，每條恰好一次 instruction fetch，另有 30 次 data references；不計投機取指或額外訪存。分離式的 I-cache 有 2 次 miss、D-cache 有 3 次 miss，故 miss rates 分別為 `2/100=2%` 與 `3/30=10%`。

若每次 miss 均造成互不重疊的額外 80 cycles，且 base CPI 不含這些停頓，追加 CPI 為 `(2 + 3)×80/100 = 4`。以每條指令計，也可寫成 `1×0.02×80 + 0.3×0.10×80 = 4`。

若另一個合併式 cache 在此 trace **也恰好**有 5 次 miss，則其 miss rate 以全部 130 次 references 為分母：`5/130`；每條指令有 `1.3` 次 references，追加 CPI 是 `1.3×(5/130)×80 = 4`。這只是分母換算，並不保證兩種配置的 miss 數相同；單埠合併式 cache 的取指／資料競爭也須另計，且不可重複計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# AMAT 的數字例

令 L1 hit time 為 1 cycle、miss rate 為 6%、main-memory miss penalty 為 80 cycles：

$$\text{AMAT} = 1 + 0.06 \times 80 = 5.8 \text{ cycles}.$$

若 base CPI 是 1.4，instruction miss rate 2%，每 instruction 有 0.25 次 data references，data miss rate 8%，則 memory stalls 對 CPI 的追加為

$$0.02(80) + 0.25(0.08)(80) = 1.6 + 1.6 = 3.2,$$

總 CPI 是 4.6。若 clock 為 2 GHz，10 億 instructions 的 CPU time 是

$$10^9 \times 4.6/(2 \times 10^9) = 2.3 \text{ s}.$$

Data misses 只在 data reference 發生，不能對所有 instructions 直接乘 data miss rate。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 更靈活的 block placement

Associativity 把 cache blocks 分成 sets：

$$N_{\text{sets}} = \frac{N_{\text{blocks}}}{E},$$

其中 $E$ 是 way count。Direct-mapped 等於 $E = 1$；fully associative 只有 1 set；set-associative 是兩者之間。

若總 data capacity 不變，associativity 每加倍一次，set 數減半、index 少 1 bit、tag 多 1 bit、comparators 約加倍。較多 ways 可降低 conflict misses，但可能拉長 tag compare 與 output select path。

Tag storage 也增加。64 KiB 例子中 direct-mapped 與 4-way 的 tag 陣列不同，不能說資料容量相同就硬體相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Locating a block 與 replacement

Set-associative hit 的步驟是：

1. Index 選 set。
2. 同時比較該 set 所有 ways 的 valid 與 tag。
3. 以 match/way select 讀出對應 data。
4. 若多個 match，表示 cache state 或協議錯誤，而不是任選一個。

Replacement choices 包含 random、LRU 與近似 LRU。2-way LRU 只需一個 recency bit；高 associativity 的 exact LRU state 快速成長，實作常改為 pseudo-LRU。

```
2-way set:  [A, B], recent=A
access B  -> recent=B
miss      -> victim A
```

Victim selection 不影響 architectural correctness，但可明顯改變 miss rate。Coherence cache 的 replacement 還要處理資料 owner 與 invalidation。

# Multilevel caches 與 AMAT 的分母

令 $T_\ell$ 是第 $\ell$ 層被存取時的 hit time（L1 miss 後才付 L2 hit time），$M_\ell^{\text{local}}$ 是送到該層的 references 中未在此層命中的比例，$P_{\text{mem}}$ 是最後仍 miss 後額外付出的 memory penalty。兩層模型是

$$\text{AMAT}_1 = T_1 + M_1 T_2 + M_1 M_2^{\text{local}} P_{\text{mem}}.$$

代入 $T_1 = 1$、$M_1 = 0.04$、$T_2 = 8$、$M_2^{\text{local}} = 0.25$、$P_{\text{mem}} = 100$：

$$\text{AMAT} = 1 + 0.04(8) + 0.04(0.25)(100) = 1 + 0.32 + 1.00 = 2.32.$$

| Quantity | 分母 | 本例 |
|---|---|---|
| $M_1$ | 所有 L1 references | 0.04 |
| L2 local miss $M_2^{\text{local}}$ | 只算 L1 misses | 0.25 |
| L2 global miss $M_1 M_2^{\text{local}}$ | 所有 L1 references | 0.01 |
| L2 hit service fraction | 只算 L1 misses | 0.75 |

# Multilevel caches 與 AMAT 的分母 （續）

若 L1 misses 中有 75% 由 L2 命中，其成本是 8 cycles；剩下 1% 的全部 references 再付 100 cycles。把 25% 直接乘 100 會把 local denominator 當 global denominator，得到錯誤的 26.32 cycles。

若另定 $P_{2,\text{miss,total}} = T_2 + P_{\text{mem}} = 108$，它是「已知 L2 也 miss」時，從開始 L2 access 到 memory 回覆的總服務時間；L2 hit 仍只需8。因此必須條件平均：

$$\text{AMAT}_1 = T_1 + M_1\left[(1 - M_2^{\text{local}})T_2 + M_2^{\text{local}} P_{2,\text{miss,total}}\right] = 1 + 0.04[0.75(8) + 0.25(108)] = 2.32.$$

只有把 $\overline{P}_{1,\text{miss}}$ 明定為「每次 L1 miss 的期望服務成本」，即 $T_2 + M_2^{\text{local}} P_{\text{mem}} = 33$，才可簡寫 $\text{AMAT}_1 = T_1 + M_1 \overline{P}_{1,\text{miss}} = 1 + 0.04(33)$。把條件於 L2 miss 的108直接乘 L1 miss rate會錯得5.32；penalty 是否包含L2時間，不能取代對L2 hit/miss的條件區分。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 5.4 Virtual memory 的兩個目的

Virtual memory 將 process 的 virtual address space 映射到 physical memory。它讓每個 process 可以在一致的虛擬位址配置下執行程式，同時由 page table 與 privilege 控制避免任意互相讀寫。

第二個目的是擴大可用容量。Main memory 像 disk-backed storage 的 cache，只保存活躍 pages；page fault 時再把需要的 page 載入。

Modern systems 最關鍵的價值通常是 protected sharing 與簡化 allocation。歷史上手動 overlay 由 programmer 安排程式片段，virtual memory 把大部分搬移工作交給 OS 與硬體。

Virtual memory 不等於「把資料無限加速」。它提供位址空間與保護；真正效能仍取決於 page residency 與 locality。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Page table 與位址翻譯

設 page size 是 4 KiB，則 page offset 有 12 bits。32-bit VA 的 top 20 bits 是 VPN；32-bit PA 的 top bits 是 physical page number（PPN）。

Page table 的 index 是 VPN，entry（PTE）通常包含：

- valid/present。
- PPN 或 disk location。
- read、write、execute 與 user/kernel permission。
- dirty 與 reference bits。
- 可能包含 cacheability/coherence 屬性。

Translation 是

$$\mathrm{PA} = \mathrm{PTE.PPN} \parallel \mathrm{VA.page\ offset}.$$

Offset 不翻譯，因為 page 與 frame 內部配置相同。

Page table entry 的完整欄位依 ISA/OS 而異；本表是教學共同模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Page fault 不只是一種 miss

TLB miss 表示硬體 translation cache 找不到 entry，page 可能仍在 memory。Page fault 則表示所需 page 不在 memory，或 access 不符合權限，OS 需要介入。

以下僅示範合法且可恢復的 not-present fault；權限違規須先判斷是否存在合法修復方式，不能直接照此載入：

1. 保存 faulting VA、access type 與可恢復 state。
2. OS 找到 free frame 或選 victim。
3. 若 victim dirty，先依其 backing 與同步規則保存內容，再重用 frame。
4. 依 mapping 從 backing storage 讀入、建立 zero-filled page，或以其他合法方式提供內容。
5. 更新 page table，必要時 invalidate stale TLB entries。
6. 重新執行 faulting load/store。

需要從backing storage調入的major page fault以儲存I/O時間計，遠高於cache miss；demand-zero、copy-on-write等minor fault可能只配置或複製memory，不一定做disk I/O。OS 優先降低 fault frequency，而不是只追求 perfect replacement。

# Page fault 不只是一種 miss （續）

Store page fault 不得在 page 保護或 space 確認前留下 partial architectural write。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Virtual memory 的 writes

Virtual memory 幾乎使用 write-back。若每次 store 都等 backing storage，成本會達數百萬 cycles；write buffer 無法吸收這種延遲。

任何 word write 都設 page 的 dirty bit。Page eviction 時只有 dirty page 需要寫回；clean page 可直接丟棄。這與 cache dirty bit 的邏輯相同，但 transmission unit 是整個 page。

Invalid PTE 可能代表 page 未配置、已 swap、保護違規或 file mapping 的特殊狀態。OS 以 fault address、error code 與 VMA/page-table 狀態決定處理方式。

Dirty 表示「不與 backing copy 相同」，不一定代表最後一次操作是寫入；OS 可在 checkpoint 後清除該 bit。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Making translation fast：TLB

TLB（translation-lookaside buffer，位址翻譯旁看緩衝）是 page-table entries 的小型 cache。Entry 可包含 VPN tag、PPN、valid、dirty、access permissions 與 ASID。

TLB hit 可由 VPN 快速得到 PPN。TLB miss 不一定表示 page fault：page 可能已在 memory，只需 page walk 並 install TLB entry。

TLB 通常 fully associative 或高 set-associative，因為容量小而 miss cost 重要。TLB entries 的總數遠少於 page-table entries，所以需要 tag；page table 則由 VPN 完整索引。

TLB dirty bit 要隨 PTE 更新同步；context switch 可能 flush TLB，或用 ASID 區分不同 process。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# TLB miss、兩層 page walk 與 cache 查詢

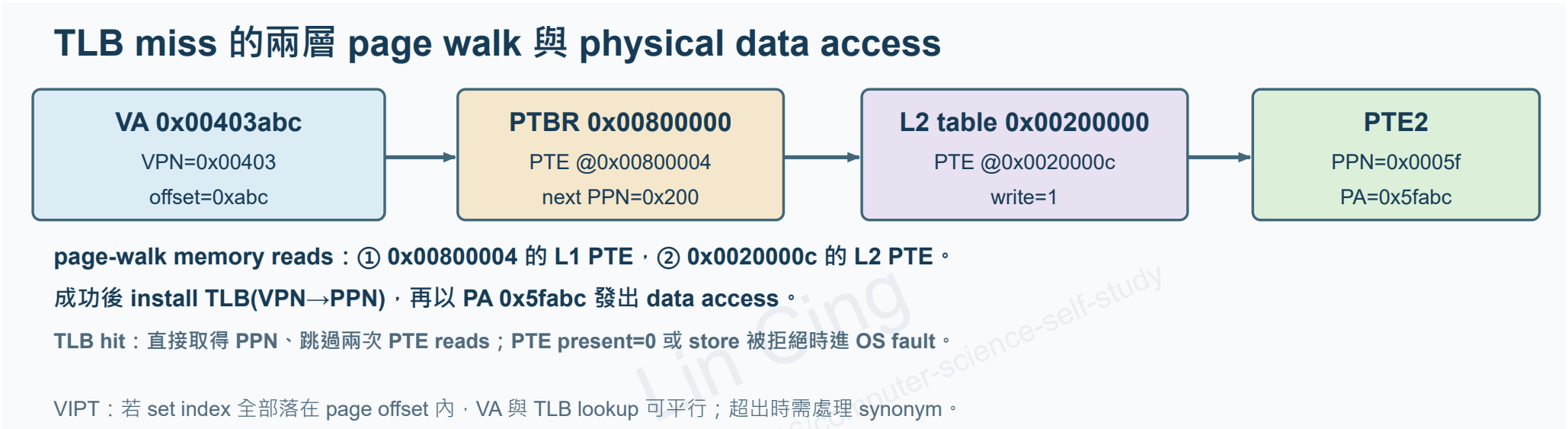
沿用 32-bit VA、4 KiB pages、PTE 4 bytes、每層 1024 entries。令 `VA=0x00403abc`：

$$\mathrm{VPN} = 0x00403, \qquad \mathrm{offset} = 0xabc.$$

兩層各 10 bits，所以 level-1 index 是 `0x001`、level-2 index 是 `0x003`。以下 PTE contents 是明定的教學輸入：

| Walk step | Address calculation | Read/decision |
|---|---|---|
| 1 | PTBR `0x00800000` + `0x001×4` | read L1 PTE at `0x00800004` |
| 2 | L1 PTE PPN `0x00200` forms table base `0x00200000` | read L2 PTE at `0x00200000 + 0x003×4 = 0x0020000c` |
| 3 | L2 PTE has `present=1`、`write=1`、`PPN=0x0005f` | install TLB mapping `0x00403→0x0005f` |
| 4 | `PA=(0x0005f<<12)\|0xabc=0x5fabc` | data access at `0x5fabc` |

# TLB miss、兩層 page walk 與 cache 查詢 （續）




TLB miss 的成本不是一個抽象常數。在本例沒有 page-walk cache、需要走完兩層的假設下，須讀取兩個 PTE，再加上資料 cache 查詢與可能的 fill；這些 PTE 存取仍可能 hit 或 miss 一般 cache。若 page-table walk 的 intermediate PTE 有專用 cache，實際 access count 必須重新數。

# TLB miss、兩層 page walk 與 cache 查詢 (續)

TLB hit 的 VA `0x00403abc` 直接得到 PPN `0x0005f`，跳過 L1/L2 PTE reads。TLB miss 但兩個 PTEs 都 valid 是 translation miss；若 walk 遇到 PTE `present=0`，則是 not-present page fault。若 `present=1`、`write=0` 而 access 是 store，結果是 protection fault。三者的 hardware path 與 recovery 不同。

對 VIPT（virtually indexed, physically tagged）cache，未翻譯的 page offset 可在 TLB 查詢時提供 set index。簡單安全配置要求 **block-offset bits + index bits <= page-offset bits**。令 block 大小 B、sets 數 S、page 大小 P，條件 BS<=P，等價於 `data capacity / ways <= page size`。只比較 index 欄位寬度與 page-offset 寬度會漏算 block offset。

64 KiB、4-way、64-byte blocks 有256 sets。Block offset佔bits5:0，index佔bits13:6；6+8=14>12，不符合4 KiB page的條件。VA `0` 與 `0x1000` 若映到同一實體頁的相同offset，分別選set0與64，可能形成同一PA的兩份synonyms（別名）副本。改成16-way，則64sets且6+6=12，別名會選同一set。

# TLB miss、兩層 page walk 與 cache 查詢 (續)

允許index超出page offset時，須另有page coloring、synonym偵測/維護，或改用PIPT。若set由VA[13:6]選取，卻只比PIPT式的PA[31:14]，就漏比physical bits13:12，可能把不同PA誤認為hit。比較完整20-bit PPN可避免此false hit，卻仍不能單靠tag comparison消除同一PA在不同sets的重複副本。18-bit PIPT tag不能直接當作此VIPT的完整physical識別條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Protection 與 context switch

Page translation 同時提供 protection。PTE 的 user/supervisor、read/write/execute 權限決定 access 是否合法。權限必須出現在 PTE 與 TLB entry；否則 TLB hit 可能繞過正確檢查。

Context switch 時若新 process 使用相同 virtual page number，舊 TLB entry 可能被誤用。解法包括：

- context switch 時 flush TLB。
- TLB tag 加入 ASID（address-space identifier，位址空間識別碼）。

ASID 讓不同 processes 的 translations 共存，降低 switch 後的 TLB misses。OS 仍需管理 ASID 重用與 invalidation。

Cache 若以 physical address 索引，通常不會因相同 VA、不同 PPN 產生同名問題；虛擬定址 cache 還需 alias/context 處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# TLB miss、page fault 與恢復

下表中「層級」描述事件來源，不是所有系統都用相同 exception code：

| 事件 | 硬體發現 | 下一步 | 是否重新執行 instruction |
|---|---|---|---|
| Cache miss | tag 不 match/valid=0 | lower-level fill | 通常是 |
| TLB miss，PTE valid | TLB 無 match | hardware/OS page walk | 通常是 |
| Page fault | PTE not present | OS 搬入 page | 通常是 |
| Protection fault | permission 不符 | 終止、signal 或 OS 處理 | 依語意 |
| Undefined VA | page table 無 mapping | OS fault | 依語意 |

若 instruction 已修改 register 或部分 memory 才 fault，pipeline 需要 precise exception 或可恢復 state。

TLB miss 與 page fault 共用 address register 不表示同一個 exception。必須追查 PTE 結果。

## 5.5 四個共同問題

Cache、TLB 與 virtual memory 都可以用四個問題描述：

1. Block/page 可以放在哪裡？direct、set-associative 或 fully associative。
2. 如何找到？index、parallel tag search 或 lookup table。
3. Miss 時替換誰？random、LRU、pseudo-LRU 或 OS policy。
4. Write 如何傳播？write-through、write-back、allocate policy。

Table lookup 可以減少 comparison hardware，但需要額外 memory access。Virtual memory 接受 page-table overhead，因為 miss cost 極高，且完全關聯式 placement 給 OS 最大配置彈性。

「fully associative 最好」缺少 hit-latency 與 comparator 成本；以下以 fully associative LRU 作 shadow 定義 3C，因此該基準本身沒有這個定義下的 conflict miss，仍可能有 compulsory 與 capacity misses。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 三類 miss：先固定比較基準

以下以相同 block size、相同 data capacity、每筆存取都更新的 fully associative LRU shadow 為基準。先確認實際 cache 發生 miss，再分類：

| 類型 | 這次 miss 的判定條件 | 常見改善方向 | 可能代價 |
| --- | --- | --- | --- |
| Compulsory（首次存取） | 這個 block 從未被存取 | 較大 block、prefetch | 流量、pollution |
| Capacity（容量） | 已存取過，shadow 也 miss | 增大容量、改善 locality | hit time、成本 |
| Conflict（映射衝突） | 已存取過，shadow hit | 增加 associativity | 比較與選取成本 |

這是對指定 trace 與比較基準的逐筆分類。第一次碰到 block 的 miss 優先列為 compulsory；「工作集大於容量」或「本次換掉同一 line」都不足以判定其餘每一筆 miss。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 用同一串存取區分三類 miss

固定 32-bit byte addresses、64 B data capacity、16 B blocks，cache 初始為空，只有 read、無 prefetch 或 coherence invalidation。實際 cache 是 4-line direct-mapped；比較基準是同樣容納 4 blocks 的 fully associative **LRU shadow cache**。

```
block sequence: 0, 4, 0, 1, 2, 3, 4, 0, 1
byte addresses: 0x0000, 0x0040, 0x0000, 0x0010, 0x0020,
                0x0030, 0x0040, 0x0000, 0x0010
```

每次都讀 block 內 offset 0。DM 的 offset 為 4 bits、index 為 2 bits、tag 為 26 bits；`line=block mod 4`、`tag=floor(block/4)`。Block 0 與 4 分別具有 tag 0 與 1，卻同樣競爭 line 0。Shadow 的索引不採此規則，四個位置可放任意 blocks。

先查 actual cache：若 hit 就不屬於 3C；若 miss，第一次碰到此 block 是 compulsory，其餘才以 shadow 的 hit/miss 判為 conflict/capacity。**兩個 cache 都處理每一筆存取，包括 actual hit**，且 LRU hit 也會更新最近使用次序。

# 第三次讀取：容量還有空位，卻已發生衝突

同容量：**4 blocks × 16 B = 64 B**；初始皆空

### #1 block 0 (0x0000)

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | — | — | — |

**DM miss / FA miss**

首次碰到此 block

### #2 block 4 (0x0040)

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 4 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | 4 | — | — |

**DM miss / FA miss**

首次碰到此 block

### #3 block 0 (0x0000)

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 4 | 0 | — | — |

**DM miss / FA hit**

conflict：0 曾被 4 淘汰

DM 的四列是固定 line 編號；FA 圖的四格則依 LRU → MRU 排列，不是固定 way 編號。第 2 次讀 block 4 時，DM 必須淘汰 line 0 的 block 0；FA 尚有空位，因此兩者可同時保留。第 3 次重訪 block 0，DM miss、shadow hit，這一次才是 conflict miss。

# 第 1–5 次：先看 compulsory 與 conflict

表內為存取**後**的狀態，– 表示 invalid。H/M 是本次 hit/miss；FA 格內依 LRU → MRU 排列。

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM 分類 |
|---|---|---|---|---|---|---|
| 1 | 0 | `[0,–,–,–]` | M | `[0]` | M | compulsory |
| 2 | 4 | `[4,–,–,–]` | M | `[0,4]` | M | compulsory |
| 3 | 0 | `[0,–,–,–]` | M | `[4,0]` | H | conflict |
| 4 | 1 | `[0,1,–,–]` | M | `[4,0,1]` | M | compulsory |
| 5 | 2 | `[0,1,2,–]` | M | `[4,0,1,2]` | M | compulsory |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 第 6–9 次：分清 capacity 與 hit

第 6 次後 shadow 含 4 個 blocks；繼續逐筆更新，而不是重新從 empty 開始。

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM 分類 |
|---|---|---|---|---|---|---|
| 6 | 3 | `[0,1,2,3]` | M | `[0,1,2,3]` | M | compulsory |
| 7 | 4 | `[4,1,2,3]` | M | `[1,2,3,4]` | M | capacity |
| 8 | 0 | `[0,1,2,3]` | M | `[2,3,4,0]` | M | capacity |
| 9 | 1 | `[0,1,2,3]` | H | `[3,4,0,1]` | M | hit |

DM 共 8 misses：5 compulsory、1 conflict、2 capacity。第 7、8 次是重訪且 shadow 也 miss；不能只因 DM 在 line 0 發生替換，就把它們都列為 conflict。

# Shadow 的 miss 不一定是 actual 的 miss

### #9 讀 block 1（byte address 0x0010）

**Direct-mapped**

固定 line 0 / 1 / 2 / 3

| block 0 | **block 1** | block 2 | block 3 |
|---|---|---|---|

**HIT：block 1 一直保留在 line 1**

狀態不變；不是任何一種 miss。

#7、#8 只替換 line 0。

**Fully associative LRU shadow**

存取前：LRU → MRU

| block 2 | block 3 | block 4 | block 0 |
|---|---|---|---|

**MISS：#8 已淘汰 block 1**

淘汰 2 → 新次序 [3, 4, 0, 1]

shadow 仍須更新，但 actual 不計 miss。

第 9 次是 **actual hit、shadow miss**，不計入實際 cache 的任何 miss 類別。全段 DM 與 FA 各有 8 misses：第 3 次的 DM miss／FA hit，恰與第 9 次的 DM hit／FA miss 抵銷。總數相減雖為 0，仍有一次 conflict；必須逐筆判定。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 改成 2-way：同容量不等於同樣的替換結果

仍為 64 B、每 block 16 B，2-way 時有 2 sets，`set=block mod 2`，各 set 各自維護 LRU。全段 hit/miss 為 `M M H M M M M M H`，共 7 misses。第 9 次前 set 0 為 `[4,0]`、set 1 為 `[1,3]`（皆由 LRU 排至 MRU），所以 block 1 仍 hit。

| 設計 | hit 次數 | miss 次數 | hit time H | 總存取時間（額外 miss penalty = 20 cycles） |
|---|---|---|---|---|
| DM | 1 | 8 | 1 cycle | `9×1+8×20=169 cycles` |
| 2-way LRU | 2 | 7 | 1.25 cycles | `9×1.25+7×20=151.25 cycles` |

此表假設每次存取皆付 H，miss 再額外付 penalty，且全部序列化、沒有重疊。本例中 2-way 較快的條件是 `9×1.25+7P < 9×1+8P`，即 `P>2.25 cycles`。

FA 允許的 placement 更多，但 **LRU 不是最少 miss 的最佳替換演算法**。本例 2-way 反而少一次 miss；不能把「可放的位置更多」誤解成「任意替換策略對每一串存取都較好」。

# 5.6 Virtual machines

System virtual machine 提供一個 guest 可執行完整 OS 的 ISA-level 環境。VMM（virtual machine monitor，虛擬機器監控器）隔離 guests，並控制 privileged state、translation、interrupts 與 I/O。

Guest 以為自己掌握 machine，實際上 privileged operations 必須由 VMM 介入或安全地直接執行。Virtualizability 取決於 ISA 是否讓 sensitive privileged state 都能被 trap 或由硬體管理。

Virtual memory 在 VM 中可能有三層：guest virtual memory、guest physical/real memory 與 host physical machine memory。

Java VM 之類的 process-level VM 不提供完整系統 ISA 與 device 隔離；不要和 system VM 混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# VM 的 translation 與 I/O

沒有硬體 nested paging 時，VMM 可用 shadow page table 把 guest VA 直接映射到 host PA，並保護/追蹤 guest 對其 page tables 的修改。Shadow update 與 TLB invalidation 是 overhead。

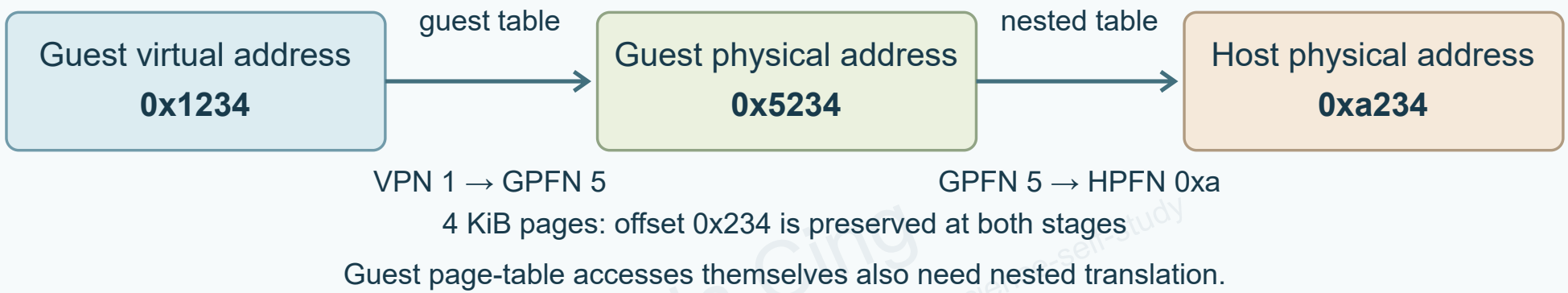
有 nested paging 時，硬體可執行兩層 walk，減少 VMM traps，但每次 TLB miss 可能多層存取。

I/O virtualization 特別複雜，因為 device、DMA、interrupt、queue 與 driver 都要隔離。VMM 可用 emulated device、passthrough 或硬體 I/O MMU。

本例的 x86/MIPS 歷史實作是本章案例範圍；今日硬體支援更豐富，結論仍以可 trap、可隔離與成本判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 雙層分頁：一次位址翻譯的實例




假設兩層皆用 4 KiB page，權限允許且所有映射有效。Guest 程式發出 GVA `0x1234`：guest VPN 為 `1`，offset 為 `0x234`。Guest page table 把 VPN `1` 映射到 guest physical frame `5`，得到 GPA `0x5234`。

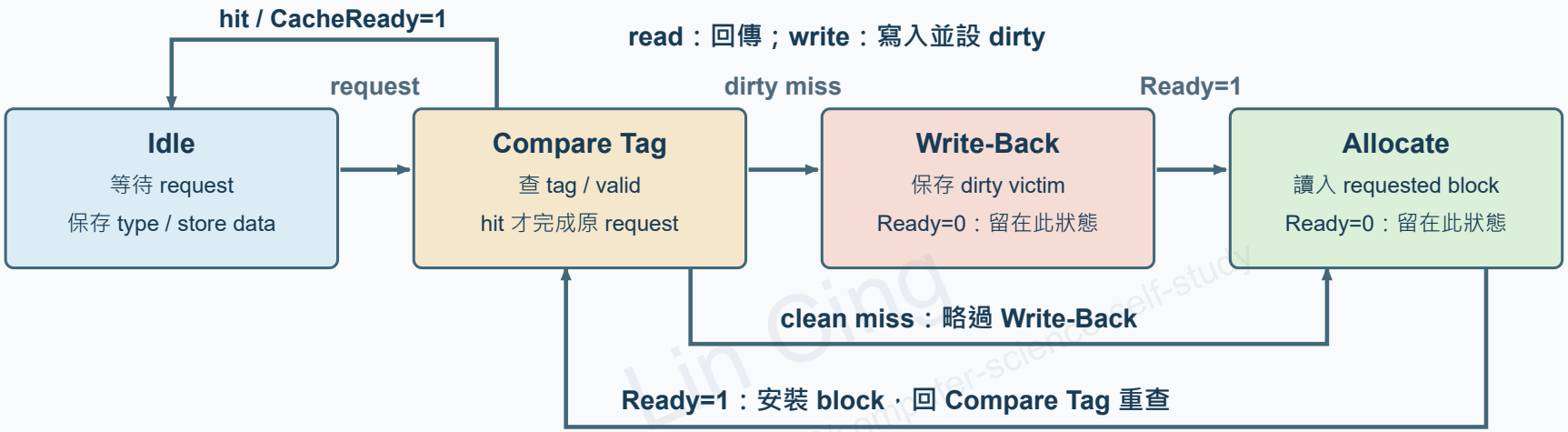
# 巢狀翻譯的額外查表

Nested page table 再把 guest frame `5` 映射到 host frame `a`，得到 host PA `0xa234`；兩次翻譯均保留 offset。Guest page table 自身位於 guest physical memory，因此讀取 guest PTE 時，也要把該 PTE 的 guest physical address 經 nested table 翻譯。TLB 或 walk cache 若命中，實際讀取次數會減少，不能一律假設固定成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.7 Cache FSM 控制

## Write-back cache：安裝後重查，完成 request 才回 Idle




Ready=0 時 Write-Back / Allocate 自迴圈；Idle 無 request 亦保持。

**Dirty write miss：Compare → Write-Back → Allocate → Compare → 寫入 store data → Idle。**

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 5.7 Cache FSM 控制 （續）

本章案例簡化 write-back cache controller 有四個 states：

1. Idle：等待 processor request。
2. Compare Tag：判斷 hit 或 miss；write hit 設 dirty。
3. Write-Back：dirty victim 寫回 memory，等待 `Ready`。
4. Allocate：fetch requested block，等待 `Ready`，完成後重查 tag。

Memory latency 可變，因此 FSM 以 `Ready` 而非固定 cycles 等待。未 assert 的 control signal 預設 deasserted；multiplexer select 仍要指定有效 value。

Write-back 與 Allocate 的 ordering 必須保證 dirty victim 在 frame 被覆寫前保存。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# FSM transition trace

以下 trace 對應 clean read miss 後 hit：

```
cycle 1: Idle + request -> Compare Tag
cycle 2: miss + victim clean -> Allocate
cycle N: memory !Ready -> stay Allocate
cycle N+1: Ready -> Compare Tag
cycle N+2: valid && tag hit -> set cache-ready -> Idle
```

Dirty write miss：

```
Compare Tag -> Write-Back -> Allocate -> Compare Tag -> Idle
```

Controller 也需要保存原 request type 與 store data；Allocate 完成後不能把 write 誤當成 read。

Early restart、critical-word-first 或 write buffer 會增加 concurrency，也增加 state 與 correctness obligations。

## 5.8 Cache coherence

共享記憶體多處理器使用 private caches，因此同一 physical address 可能有多份 cached copies。Coherence 規定已排序的 read 可看到哪個 value，以及同一位置的 writes 如何被序列化。沒有 synchronization 的 data race 可以產生依 execution order 而定的結果；read 看到舊值本身不證明 coherence protocol 壞掉，仍需看該 read 與 write 是否由 consistency model 排序。

Coherence 至少保證：

1. 同一 processor 在 program order 先寫同一位置，且其間沒有其他 processor 對同址的有效 write 時，後續 read 看到自己的 write。
2. 對同一位置的 writes 被所有 processors 以相同順序觀察。
3. 若 synchronization 已把 read 排在某個 write 之後，且其間沒有更晚的同址 write，該 read 回傳該 write 的 value。

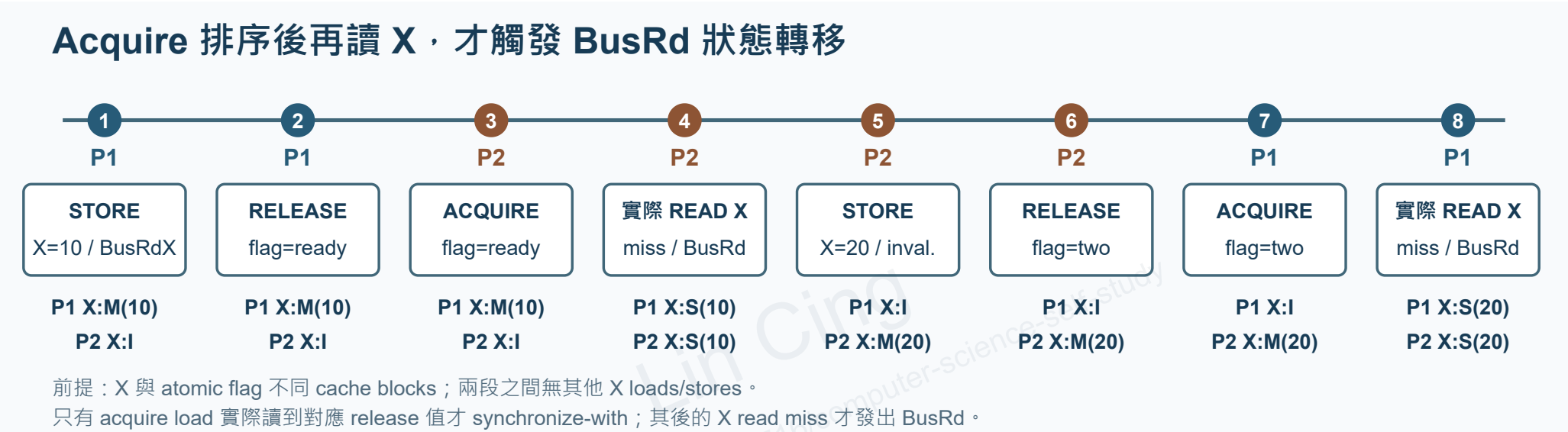
Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 5.8 Cache coherence （續）

第三點需要 memory consistency model 才能精確定義跨 thread 的「後續」。Coherence 管同一位置及其可見 value，consistency 管跨位置與 synchronization 的 ordering。只有當 atomic operation 依規格建立 synchronization，而且該關係涵蓋相互衝突的 accesses 時，才排除那一組 data race；它不會修正其他未排序存取，也不會把任意舊值讀取自動變成違規。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

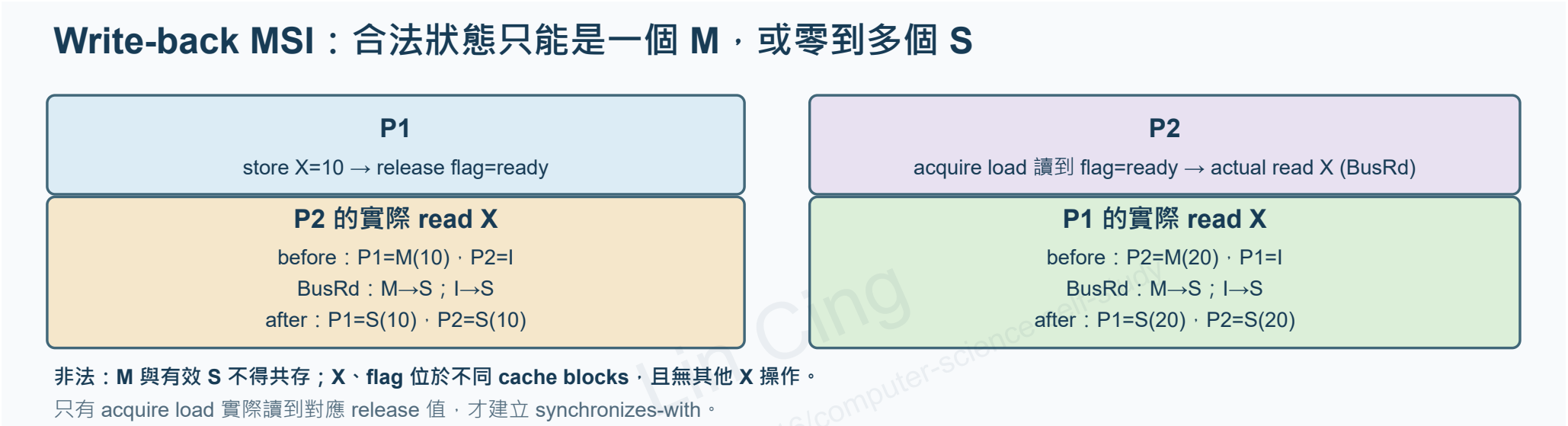
# 其他 write 介入後的 Write-back MSI 時序




先看 P1 寫入 10，再發布 `ready`；P2 的 acquire 必須讀到這個 release 值，才能把先前的寫入排序在 P2 後續讀取之前。Acquire 本身不讀 X；P2 之後的 X read miss 才發出 BusRd，由 P1 的 M owner 提供 10。

P2 隨後寫入 20 並發布 `two`；P1 成功 acquire 後再讀 X，才能沿相同推理取得 20。本圖假設旗標與 X 位於不同 block，且兩段之間沒有其他 X 存取；共享 block 仍可同步，但會有額外的 coherence traffic。

# MSI 狀態不變量與 acquire/read 的差別




工作例明定 x 與 atomic `flag` 位於不同 cache blocks，且兩段程式之間沒有其他 X 操作。P1 store `X=10`，再以 release store 寫 `flag=ready`；P2 以 acquire load 實際讀到這個 released value 時才建立 synchronizes-with，接著**實際 read X**。P2 store `X=20`，再 release `flag=two`；P1 的 acquire load 實際讀到 `two` 後，才再次**實際 read X**。

# MSI 狀態不變量與 acquire/read 的差別 （續）

| 事件（program order） | P1 X | P2 X | flag 與 ordering 關係 |
|---|---|---|---|
| 初始 | X:I | X:I | X 未 cached；flag 尚無 release/acquire 關係 |

| 事件（program order） | P1 X | P2 X | flag 與 ordering 關係 |
|---|---|---|---|
| P1 store X=10；write miss / BusRdX | X:M(10) | X:I | P1 在自身 cache 完成 write；尚未建立與 P2 的跨 thread happens-before |
| P1 release flag=ready | X:M(10) | X:I | release 排序 P1 先前的 X store 與 atomic flag write；不要求把 dirty X flush 到 memory |
| P2 acquire flag=ready | X:M(10) | X:I | acquire load 實際讀到該 released value 才 synchronize-with；**不讀 X，也不改變 X cache state** |
| P2 acquire flag=ready 後，實際 read X；read miss / BusRd | X:S(10) | X:S(10) | P1 的 M owner 供給 10；P1 M→S，P2 I→S，memory 更新為 10 |
| P2 store X=20；BusRdX/upgrade | X:I | X:M(20) | P2 取得 exclusive permission，P1 的有效 copy 失效 |

# MSI 狀態不變量與 acquire/read 的差別 （續）

| 事件（program order） | P1 X | P2 X | flag 與 ordering 關係 |
|---|---|---|---|
| P2 release flag=two | X:I | X:M(20) | release 排序 X=20 與 atomic flag=two |
| P1 acquire flag=two | X:I | X:M(20) | acquire load 實際讀到 `two` 後才 synchronize-with；**P1 尚未讀 X** |
| P1 acquire flag=two 後，實際 read X；read miss / BusRd | X:S(20) | X:S(20) | P2 的 M owner 供給 20；P2 M→S，P1 I→S，memory 更新為 20 |

Write-back MSI 的不變量是同一 block 最多只有一個 M；M 存在時，任何其他 cache 都不能保有有效 S。本表的合法 read miss 都由原 M owner 提供資料，並同時把 owner 降為 S。`acquire flag` 不是 `load x`；真正讀 X 的 load 才觸發 BusRd 與上述狀態轉移。若 x 與 `flag` 共用 cache block，false sharing 另會觸發該 block 的 coherence traffic；本表不把那個額外效果混入 X 的狀態。

# Snooping 與 write-invalidate

Snooping protocol 讓每個 cache 觀察 shared interconnect 的 block request，沒有 central directory。它適合 broadcast 可負擔的系統，processor 數增加後 traffic 與 scalability 成問題。

Write-invalidate protocol 在寫入前取得 exclusivity，使其他 cached copies invalid。常見簡化 state：

- I：Invalid。
- S：Shared，可讀且可能有多份。
- M：Modified，只有此 cache 有有效 copy，main memory 可能 stale。

Read miss 可能在 S 或 M 找到 supplier；write hit 在 S 需發 invalidate；M owner 被另一 read 觀察時須提供最新 data 並調整 state/write-back policy。

BusRd/BusRdX、cache-to-cache transfer 與 writeback 的行為依 protocol；state table 必須完整指定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.9 Cache controller 的設計

Cache controller 是處理 variable-latency memory 的狀態機。`Idle` 接收 request，`Compare Tag` 判定 hit/miss，`Write-Back` 保存 dirty victim，`Allocate` 等待 lower-level block。Memory 尚未回應時 `Ready=0`，完成時安裝 block、更新 valid/tag 並清除新 block 的 dirty，再回 `Compare Tag` 重查。只有重查命中並完成原 read 或 write 後，才 assert `Cache Ready`；原 request 是 write 時還要寫入保存的 store data 並設 dirty。

Controller 必須保存 request type、store data、fault/retry 所需的 state，以及 victim ownership。`Write-Back` 必須在 victim slot 被覆寫前完成，或先把 victim 安全放入有完成保證的 buffer；`Allocate` 不可越過這個 ordering condition。

可檢查的設計不變量包括：任何 valid cache line 的 tag 唯一指向其 data、dirty line 在被無備份覆寫前已保存、response 只在 block/data ready 後產生，以及 reset/error 不得留下半完成 transaction。這些 invariants 可用 assertion 或 model checker 測試。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 5.10 Real systems

本章案例 AMD Opteron X4（Barcelona）與 Intel Nehalem 案例都把 larger caches、TLB 與 memory controller 放在 die 上。Dies 有私人 L1/L2、shared L3 與多層 TLB，降低 off-chip communication，但增加 area、power 與 coherence 複雜度。

Nehalem 案例的公開教材數據是每 core 私人 instruction/data cache、L2，以及四 cores 共享 L3。本章案例另一個關鍵是 lower-level caches 可由 data outside this core fill，並可採 victim cache 式關係。

這些 2008 年尺寸用來理解 cache sharing、inclusion 與 on-die traffic tradeoff，不是今日 purchase benchmark。

若 L3 non-inclusive，L2 miss 仍可能由 L3 hit；不能假設上層缺失一定能由直接下層找到。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.11 Fallacies and pitfalls

第一類錯誤是忽略 byte addressing。32-byte data cache、4-byte blocks 有 8 blocks；byte address 36 先整除 4 得到 block 9，再 mod 8 得到 block index 1。

第二類錯誤是把 block size 當成 block count。容量除以 block size 才是 blocks，不是把 address 除以 cache capacity。

第三類錯誤是把 local/global miss rates、instruction/data references 或 ns/cycle 混用。

第四類錯誤是只看 hit rate。更大的 L1 可能 miss rate 較低，但 clock path 變長，整體 AMAT 反而上升。

每題先寫單位與分母，通常可避免大半的手算錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.12 Concluding remarks

Memory hierarchy 的統一原理是 locality、block transfer、placement/search、replacement、write propagation 與 miss cost。Cache 與 virtual memory 的參數差很多，policy 結構卻相似。

多層 cache 可以把 miss rate、hit time 與 miss penalty 交給不同層最佳化。Software 也能用 cache blocking、loop ordering 與 prefetch 改善 locality，但最佳化需以測量驗證。

「增加 L3」不是萬用加速器；working set、bandwidth、coherence 與 OS 排程共同決定結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.13 Historical perspective

早期 virtual memory 與 cache 是面對有限 memory、program relocation 與 protection 的工程解。分頁、TLB、set-associative cache 與 snooping coherence 逐步成為共同設計模式。

Hierarchy 變得複雜後，硬體仍保留可觀察模型：process 看到 virtual memory，ISA 規定 memory access 語意，coherence/consistency 規範 shared-memory behavior。效能實作可以不同，architectural correctness 不能悄悄改變。

歷史數字顯示技術比例會變，但 locality、protection 與 data movement 成本不會因某一代製程技術而消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 章末驗收

完成本章後，應能：

1. 由 byte address 手工切出 tag/index/offset。
2. 對任意 trace 列 hit/miss、victim、dirty/write-back。
3. 從 AMAT 推到 CPI 與 elapsed time，並說明 stall 假設。
4. 畫出 page walk、TLB hit、TLB miss 與 page fault 的分支。
5. 對 write-invalidate protocol 寫出 state transition。
6. 說明 3Cs、local/global miss 與 consistency model 的邊界。

先完成本章獨立練習題 PDF，再以詳解 PDF 核對單位、假設與每一步。

### 參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 5. CSO course slides, memory hierarchy.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

# 章末驗收 (續)

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study